

Solo-Developer Strategy for a Full Engine With a Six-Month 2D Priority Track Using Epoch Engine as the Baseline

Executive summary and explicit assumptions

Epoch Engine already has several “expensive-to-get-right” foundations in place for a solo developer: a module-first organization, a multi-context/multi-backend runtime (OpenGL, SDL, Raylib, SFML, software, headless/noop, and an experimental Vulkan path), an editor/launcher split with a project-driven workflow, an engine-owned texture-atlas-based UI/sprite/text pipeline, an ECS-style direction, compiled C++23 scripting integrated into the build, a task-graph orientation, a time-system “spine” (fixed-step ownership, pause/resume, scaling, single-step), and a disciplined smoke/capture process that treats regressions as a first-class risk. ¹

Your updated goal is not “a 2D engine,” but “a full engine where 2D is the fastest shippable lane for the next six months,” while continuing to grow reusable systems through modular, functional, efficiency-first design. That’s feasible—if you treat 2D as a vertical slice through core engine primitives (time, resources, render submission, input, simulation, save/load, tooling), not as a pile of 2D-only hacks.

Assumptions used throughout (call these out early so the plan stays honest):

- **Primary platform:** desktop first; mobile is optional and only pursued after the 2D lane is stable. ²
- **Baseline hardware target:** Epoch docs explicitly frame support discipline around “6-core / GTX 1660 Ti-era desktops,” so the six-month 2D bar is pegged to that class of machine. ³
- **Time budget for a solo developer:** 15–25 focused hours/week (part-time serious) or ~35–45 hours/week (full-time). The roadmaps below provide both pacing options as “capacity multipliers” rather than pretending one schedule fits all. (Heuristic; not a sourced claim.)
- **Game-facing prototype requirement:** a 2D underground sandbox with **pixel/material simulation (falling-sand style), digging/destruction, animated characters, grenade time-warp VFX, and NPC AI**. The plan prioritizes these explicitly. ⁴

What Epoch Engine already provides and what it implies for the roadmap

Epoch’s README and internal docs make several architectural commitments that significantly shape “what you should do next,” because they constrain what will be maintainable in two years:

- **Multi-context + multiplexer core.** Your active architecture describes a shared context layer and a “multiplexer/window/control” spine coordinating backend windows, command queues, and render-thread work. ⁵

- **Multi-backend reality.** OpenGL is treated as the primary GPU path; software is a validation/fallback path; Vulkan is explicitly “experimental/migration,” not stable default. ⁶
- **A regression discipline already exists.** You have a documented minimum smoke suite (startup/resize/atlas upload/present/shutdown) per backend, and a capture workflow (`--capture` , `--smoke` , `EPOCH_CAPTURE_DIR`) with a commit/version cadence that bakes in “prove it works” habits. ⁷
- **Engine-owned time control is a declared pillar.** The time spine (fixed-step accumulation, pause/resume, scaling, single-step) is explicitly positioned as central ownership—not per-subsystem ad hoc timing. This is unusually valuable for your “timewarp grenade” requirement, because you can make time manipulation a supported gameplay feature rather than a shader-only trick. ⁸
- **Module-first surface and module inventory.** Your docs enumerate module families (`core.*` , `context.*` , `render.*` , `perf.*` , `ecs.*` , `atlas.*` , plus backend modules). The exported `epochengine` module imports `core.context` , atlas modules, scripting, taskgraph, updater, and `opengl.context` , which tells us where the “public engine surface” is intended to converge. ⁹
- **Project-driven runtime + compiled scripting direction.** Runtime operations doc insists the editor plays the active project/scene, and scripting is compiled with the engine/project rather than string evaluation; hot reload is acknowledged as risky and needs explicit smoke coverage. ¹⁰
- **AI data hygiene policy.** You already distinguish repo-safe AI assets vs local-only artifacts, which matters for security and distribution when you start shipping tooling that touches model artifacts. ¹¹

The practical implication: **your six-month lane should not create a parallel “2D subsystem universe.”** Instead, it should “stress-test” and harden the existing spines (time, ECS-style systems, atlas/sprite submission, project workflow, capture, save/load) by building the 2D sandbox game as the forcing function. That approach aligns with how the README frames Epoch as a single modern codebase for games, editors, tools, and pipelines. ¹²

Engine architecture patterns optimized for a six-month 2D lane without sacrificing the full-engine trajectory

This section defines the architecture patterns that best fit (a) your existing Epoch commitments and (b) your specific near-term gameplay requirements.

Rendering pipeline and frame structure

Epoch’s README hints at a modern staged renderer baseline (“visibility → surface → lighting → temporal → reconstruction → present”), with heavier techniques behind support tiers. ¹³

For a six-month 2D lane, you don’t want to build an entirely different rendering philosophy; you want a **2D specialization of the same stage model**:

- **Visibility:** coarse culling by camera bounds and chunk bounds (for your pixel world, visibility is mostly chunk-level).
- **Surface:** sprite batching + tile/pixel surface compositing. Epoch already has an engine-owned atlas and sprite pipeline; leverage this rather than adding a second UI/sprite stack. ¹⁴

- **Lighting (optional but likely necessary underground):** a 2D light grid (tile/chunk light) or signed-distance-field lighting later; keep an initial “darkness + local point lights” model to support “soldier underground” readability. (Design guidance; you can validate visually in the slice.)
- **Temporal/reconstruction:** in 2D, treat these as **post-processing hooks** rather than full temporal AA. Your “timewarp grenade” can live here as a post pass (UV displacement, radial shockwave, chromatic split, motion smear) applied via render-to-texture and a full-screen quad. Framebuffer Objects (FBOs) are the standard OpenGL mechanism for leading a post-processing chain. ¹⁵

In Epoch terms, keep OpenGL as the **golden path** for the 2D lane (because it’s explicitly the primary GPU path today), and keep the software renderer as a correctness oracle where feasible (e.g., atlas correctness, deterministic captures). ¹⁶

ECS vs OOP vs hybrid: what scales for pixel simulation + AI

Your docs already describe an “ECS-style systems” direction. ¹⁷

For your particular prototype (lots of tiny simulated elements + a few higher-level agents), data-oriented ECS is a good fit—**but only if you draw a hard boundary between:**

- **Macro entities** (player, NPCs, items, grenades, interactables), and
- **Micro cells** (pixel/material grid).

Do **not** make every cell an ECS entity. Instead, treat the grid as a specialized simulation subsystem with its own dense SoA arrays and chunk scheduling. This mirrors why archetype/SoA ECS improves cache efficiency and frame stability for large numbers of entities, while still requiring careful data layout decisions. ¹⁸

A rigorous way to frame this (and avoid “religious wars”) is to separate **state representation** from **iteration strategy**:

- ECS is excellent when your iteration is “for all entities with (Position, Velocity...) do X.” ¹⁹
- A cellular/pixel simulation is excellent when your iteration is “for all active cells in dirty chunks do local rules,” which is better represented as **chunked grids + swap buffers**, not entity tables. The GDC talk on *Noita* explicitly discusses scaling falling-sand simulation to large continuous worlds and integrating destructible rigid-body physics—strong evidence that hybridizing cellular simulation with additional physics structures is a proven path for games like yours. ²⁰

Resource management and asset pipeline: keep it boring and deterministic

Your engine already leans on an atlas-driven UI/sprite/text pipeline, and emphasizes launching from asset-bearing output directories to keep contexts consistent. ²¹

For a solo developer, the best asset pipeline is the one that:

- Minimizes editor complexity early,
- Produces deterministic outputs suitable for captures and regression comparison, and
- Allows hot iteration without turning your build into a ritual.

A pragmatic, Epoch-aligned approach:

- **Asset import stage** (external): use simple CLI tools/scripts to convert source art into engine-ready formats (atlas pages, metadata, sprite sheets, sound banks). Use Epoch's existing build/run scripts discipline as the anchor, then expand automation incrementally. ²²
- **Runtime asset DB** (internal): stable IDs/handles; versioned metadata; deterministic packing.
- **Hot reload discipline**: treat hot reload as "developer convenience" and keep it behind a *known smoke path*, matching your own doc warning ("hot reload remains a development feature and needs smoke coverage instead of trust"). ²³

Input, audio, UI, serialization: recommended minimal patterns for the six-month lane

- **Input**: keep a single "input state snapshot" per frame (or fixed tick), derived from backend events. This is naturally compatible with deterministic replays later (even if you don't build replays yet). Factorio's deterministic mindset around save/load and replay viability is a useful north star for "future you," because it shows how strict determinism unlocks testing and replay features. ²⁴
- **Audio**: prefer a low-friction, single-dependency solution early. *miniaudio* is explicitly designed as a single-file C/C++ playback/capture library with no external dependencies and broad platform support, which is very solo-friendly for the 2D lane. ²⁵
- **UI**: keep Epoch's custom atlas-based UI for the engine/editor shell (because it's already integrated), but consider adding **an optional tooling UI** path with *Dear ImGui* for debug/profiling panels if it reduces time-to-tool. ImGui is explicitly pitched for fast iteration and tooling in real-time engines, outputting vertex buffers you render in your pipeline and minimizing retained state. ²⁶
- **Serialization**: implement **versioned, deterministic serialization** early for the 2D sandbox world (chunks + entities). Factorio's dev write-up explains why deterministic save/load is surprisingly hard but hugely valuable (replay viability, ability to test save/load correctness by ensuring no observable behavior changes). ²⁴

Comparison table: architecture choices that matter most for your goals

Decision point	Option	When it wins	Biggest risk	Recommendation for Epoch's six-month 2D lane
Gameplay object model	Pure OOP inheritance	Small toy games, few entity types	"God object" hierarchies, poor cache behavior	Use OOP locally (private implementation), but avoid inheritance as the <i>world model</i> . ¹⁸
Gameplay object model	Data-oriented ECS	Many entities, many systems, predictable iteration	Complexity in system scheduling & structural changes	Use ECS for macro entities (player/NPC/items). Keep it archetype/SoA-leaning for iteration stability. ²⁷
Pixel/material simulation	ECS-per-cell	Conceptually uniform	Catastrophic scale overhead	Don't do this. Use chunked grids with dense arrays. ²⁸

Decision point	Option	When it wins	Biggest risk	Recommendation for Epoch's six-month 2D lane
Pixel/material simulation	Chunked cellular sim (CPU)	Easy debugging, deterministic, portable	CPU cost if you update too much	Start here with dirty-chunk scheduling; it's the most controllable baseline. ²⁸
Pixel/material simulation	GPU compute / ping-pong textures	Massive throughput for large grids	Debugging complexity, backend divergence	Add after you have a correct CPU version; OpenGL compute shaders exist since 4.3 and SSBOs are core since 4.3, enabling robust GPU simulation storage. ²⁹
Post effects (timewarp)	"In-line" sprite shader tricks	Cheap, simple	Hard to compose, fragile	Use an FBO-based post chain—standard render-to-texture + full-screen pass—so effects stack cleanly. ¹⁵

Designing the specific six-month 2D feature set: pixel physics, digging, timewarp grenades, animation, NPC AI

This is the heart of your ask: what a "viable" six-month 2D lane looks like **for your specific game**, not for a generic platformer.

Pixel physics and digging: a hybrid simulation model modeled after proven patterns

Games like *Noita* ³⁰ are explicitly described (and marketed) around "every pixel simulated," and the GDC talk summary highlights scaling falling-sand simulation to large continuous worlds and integrating destructible rigid bodies. ³¹

That strongly suggests a robust architecture for your "2D Minecraft-like sand sim with digging" is **two coupled simulations**:

- **Cellular material sim** (grid-based): sand, gravel, water, smoke, etc.
- **Rigid/continuous physics** (agent & props): player capsule, NPC body, grenades, rigid debris.

You can implement the coupling via **extractors**:

- Grid → physics: generate collision solids from "solid cells," either as tile colliders (fast and blocky) or contour/marching outlines (smoother but harder).
- Physics → grid: collisions deposit impulses/heat/explosions into cells; digging edits cells; explosion converts material.

The "two-system" story also shows up in community notes inspired by *Noita's* engine work, but the key primary anchor is the GDC session description itself. ²⁸

Implementation detail that makes or breaks performance: chunking + dirty propagation

For a solo engine, the performance cliff is almost always “I updated the whole world every frame.” Your baseline must be:

- **Chunked world storage** (e.g., 64×64 or 128×128 cells/chunk).
- **Dirty chunk propagation** (a chunk becomes “active” if any cell changed, or if neighbor flux requires it).
- **Fixed-step simulation budget** (e.g., N simulation steps per tick, with time spine controlling determinism). Epoch’s time spine is already positioned to own fixed-step accumulation and time scaling. ⁸

GPU acceleration is an option later. OpenGL compute shaders (core since 4.3) and SSBOs (core since 4.3) are the canonical primitives if you want a GPU material sim pipeline without jumping to Vulkan immediately.

²⁹

A real-world datapoint (non-academic but concrete) is a compute-shader falling-sand demo reporting 200+ FPS on a 1024×1024 grid with 8 sim steps per frame, which supports the claim that GPU approaches can scale extremely well once correctness is established. ³²

Timewarp grenades: treat as both a gameplay time feature and a post-process

You have two orthogonal “timewarp” levers, and you should deliberately separate them:

- **Visual timewarp (VFX):** radial distortion/shockwave, chromatic aberration, motion smear. This is best implemented as a post-processing pass using FBO render-to-texture, because it composes with other post steps and is backend-portable within your renderer model. ¹⁵
- **Simulation time manipulation:** this is where Epoch’s time spine shines. If you keep the global engine clock authoritative (fixed-step), you can implement per-entity “local time scale” by scaling integration step sizes inside specific systems for entities in a time field—without breaking the engine’s global pacing controls. Epoch’s docs explicitly plan time scaling and single-step control as first-class. ³³

A practical “six-month version” is: **global slow-mo + strong VFX**, plus a small-radius local effect that impacts only grenades/projectiles and nearby particles (so you can ship the fantasy without rewriting your whole scheduler).

Animated characters: ship with sprite sheets first, leave skeletal for later

Your engine already has an atlas manager/texture pipeline and sprite registry modules imported into the exported engine surface. ³⁴

For six months, the lowest-risk animation stack is:

- Sprite-sheet animations (frame index + fps + event markers).
- Simple blend rules (idle/run/jump/dig/throw).
- Animation events (footsteps, dig-hit, grenade throw) feeding audio and particles.

Skeletal/2D bone animation is a year-two feature unless it's core to your art style.

NPC AI: a layered approach that matches your world's constraints

You want an underground NPC soldier in a deformable world. That's basically "navigation in a map that changes under your feet," which is why simplistic static navmeshes fail.

A robust and well-sourced layered approach:

- **Tactical decision layer:** Behavior Trees (BTs) provide modular/reactive behaviors and are explicitly framed in the literature as addressing brittleness of large FSMs while enabling modularity and reactivity. ³⁵
- **Goal/plan layer (optional but powerful):** GOAP (Goal-Oriented Action Planning) is a proven approach in shipped commercial AI systems (notably described through GDC material around F.E.A.R. ³⁶ and GOAP's evolution), using A* planning on actions rather than just path nodes. ³⁷
- **Navigation layer (must handle change):** *A is the classic baseline for shortest paths, but for dynamic/replanning environments, D Lite is a canonical incremental heuristic search algorithm designed to reuse prior search work and replan efficiently.* ³⁸

For your six-month lane: implement **BT + A** first, then upgrade to *D Lite* if digging/destruction makes replanning cost too high. This sequence is "minimum viable sophistication."

Roadmaps, staffing estimates, and action plans for six months and two years

Epoch's own docs emphasize cadence and discipline: versioned commits, smoke/capture automation targets, and backend regression isolation. ³⁹

This section converts that discipline into a concrete delivery plan.

Six-month prototype feature set: prioritized MVP for your specific game

The six-month bar is a **playable vertical slice** that uses the same engine primitives you will keep long-term.

Definition of Done (six months): you can build/run from the output directory, open a project, play a scene, simulate a chunked pixel world with digging, control an animated character, throw a grenade with timewarp VFX, and fight at least one NPC type with pathfinding and basic tactical behavior. ⁴⁰

Priority	Feature	Why it's critical	Minimum viable implementation	Proof artifact
P0	Deterministic time + fixed step loop integration	Everything else depends on stable pacing	Use <code>core.time</code> spine controls (fixed-step accumulation, scaling, pause/step) as the only authority	Capture + perf stats overlay in Systems workspace ⁴¹

Priority	Feature	Why it's critical	Minimum viable implementation	Proof artifact
P0	Chunked pixel/material world + digging	Core gameplay	CPU cellular sim with dirty chunks, solid/empty/sand/water; digging edits cells	Saved world + reloaded world matches behavior ⁴²
P0	Character controller tied to grid collisions	"Soldier underground" requires it	Capsule/box controller; collide against solid cells (tile collider first)	Automated smoke scene: run/jump/dig ⁴³
P0	Rendering: tile/pixel surface + sprite batching	Must see the world at speed	Render chunk textures (dynamic update) + sprite atlas for entities	Backend smoke captures across OpenGL + software ⁴⁴
P1	Grenade + timewarp VFX	Signature mechanic	FBO post-pass shockwave + global slow-mo hook	Capture shows consistent effect from same seed ⁴⁵
P1	Animated sprites + events	"Feels like a game"	Sprite-sheet animation state machine, event hooks	Animation debug panel (ImGui optional) ²⁶
P1	NPC AI: BT + pathfinding	Gameplay loop	BT for tactics + A* grid path; replan on blocked routes	AI regression map tests ⁴⁶
P2	Save/load of chunks + entities	Stops you from losing months	Versioned serialization; deterministic load	Save/load diff test harness ²⁴
P2	Audio (steps, digging, explosions)	Feedback loop	miniaudio + simple mixer	Audio smoke clips recorded ²⁵

Two-year shipping feature set: a realistic "full engine + shipped game" scope

Two years is enough to ship a strong game and a credible engine **if you resist infinite-platform promises** and treat Vulkan/extra backends as second-order until they're forced by distribution needs. Epoch docs already caution Vulkan is not the default. ⁴⁷

A realistic two-year scope for your genre:

- **Simulation depth:** more materials + interactions (fire, smoke, temperature, liquids mixing), chunk streaming, explosion propagation.
- **Tooling:** world painting tools, material debug views, capture-based regression, save inspection.
- **AI depth:** D* Lite replanning for deformable terrain; more NPC roles; debugging visualizers. ⁴⁸

- **Performance hardening:** GPU acceleration path (compute shaders) if CPU budget fails; multi-threaded chunk updates using the task graph. ⁴⁹
- **Distribution readiness:** packaging, crash reporting, deterministic settings, license clarity. ⁵⁰

Mermaid system architecture sketch aligned to Epoch module families

```

flowchart TB
    subgraph Core["Core spine (module-first)"]
        Time["core.time (fixed-step, scaling, step)"]
        Jobs["taskgraph.* (jobs, async work)"]
        ECS["ecs.* (macro entities)"]
        Resources["resource/saveload.* (assets + serialization)"]
        Context["core.context + context.* multiplexer"]
    end

    subgraph Backends["Backends (multi-context)"]
        OGL["opengl.* (primary GPU path)"]
        SDL["sdl.*"]
        Ray["raylib.*"]
        SFML["sfml.*"]
        Soft["software.* (fallback/validation)"]
        VK["vulkan.* (experimental)"]
    end

    subgraph GameLane2D["Six-month 2D lane (the vertical slice)"]
        PixelSim["Chunked pixel/material simulation"]
        Dig["Digging/destruction tools"]
        Player["Player + animation"]
        NPC["NPC AI (BT + pathfinding)"]
        FX["Timewarp grenade (post-process + time scaling hook)"]
    end

    Context --> Backends
    Time --> PixelSim
    Jobs --> PixelSim
    ECS --> Player
    ECS --> NPC
    PixelSim --> Dig
    PixelSim --> Player
    PixelSim --> NPC
    Backends --> FX
    Resources --> GameLane2D

```

Mermaid development timeline for the six-month lane

```
gantt
    title Six-month 2D priority lane (while staying a full engine)
    dateFormat YYYY-MM-DD

    section Foundations
    Lock 2D golden path backend (OpenGL + software) :a1, 2026-04-08, 14d
    Deterministic fixed-step + capture-ready test map :a2, after a1, 14d

    section World simulation
    Chunked cell storage + dirty scheduling :b1, after a2, 21d
    Materials v1 (solid/sand/water) + digging :b2, after b1, 21d

    section Gameplay
    Player controller + grid collision v1 :c1, after b2, 21d
    Sprite animation + events :c2, after c1, 14d

    section Effects
    Grenade + timewarp post-pass v1 :d1, after c2, 14d
    Time scaling integration (global + local prototype) :d2, after d1, 14d

    section AI
    NPC v1 (BT + A* grid path) :e1, after d2, 21d
    AI debug overlays + regression maps :e2, after e1, 14d

    section Shipping hygiene
    Save/load v1 (chunks + entities) :f1, after e2, 21d
    Full smoke suite + automated captures in CI (start) :f2, after f1, 21d
```

Weekly schedule templates and time allocation chart

A solo plan fails not because of “bad code,” but because your week gets eaten by context switching. Epoch’s existing “commit pattern memory” and capture automation goals are exactly the right instinct: reduce variance, keep evidence, keep cadence. ⁵¹

Suggested weekly structure (choose one capacity tier; heuristic):

- **Part-time serious (15–25h/week):** 3 deep-work blocks + 2 shorter maintenance blocks.
- **Full-time (35–45h/week):** 4 deep-work blocks + 1 tools/regression day.

Mermaid time allocation (recommended baseline):

```
pie title Weekly time allocation (recommended)
    "Core features (simulation/render/gameplay)" : 55
    "Tooling + tests + captures" : 20
```

```
"Bugfixing + refactors" : 15
"Docs + planning + backlog grooming" : 10
```

Risk impact chart and mitigation, tied to Epoch's existing regression discipline

Your own renderer regression plan already identifies the correct instinct: define smoke scenarios, define pass criteria, and treat backend fixes as isolated where possible. ⁵²

A risk matrix for your specific project (impact and likelihood are heuristic scores):

```
xychart-beta
  title "Top risks (impact score out of 10)"
  x-axis ["Pixel sim perf cliff","Digging+collision bugs","Save/load
corruption","Multi-backend regressions","AI nav in deformable world","Hot reload
instability","Scope creep"]
  y-axis "Impact" 0 --> 10
  bar [9,8,9,7,8,6,10]
```

Mitigation mapping to existing Epoch practices:

- **Multi-backend regressions:** keep a “golden path” backend and run the documented backend smoke suite after any startup/resize/atlas/present changes. ⁵³
- **Save/load corruption:** adopt Factorio-like determinism checks (“save→exit→load→run produces no observable change”) as an automated test target. ²⁴
- **Hot reload instability:** treat hot reload as best-effort and require explicit smoke coverage before trusting it. ²³

Step-by-step task list for the next six months

This is the actionable “do the work” sequence, designed to produce playable results early while hardening engine spines.

Month one 1) Declare the 2D golden path renderer/backend combination (OpenGL + software fallback) and freeze broad backend expansion for the slice. ⁵⁴

2) Build a single canonical “2D sandbox test project” that always launches via the project-driven runtime path (no hidden sample fallbacks). ²³

3) Add a capture-based regression map: on launch, auto-run a short deterministic scene and produce a capture set using the engine-owned capture knobs. ⁵⁵

Month two 4) Implement chunked cell storage + dirty scheduling + minimal materials (solid/sand/water) + digging edits. ²⁸

5) Implement player controller against grid collisions (tile-collider first).

6) Render chunks efficiently (update only dirty chunk textures; sprite-batch entities using the existing atlas pipeline). ⁵⁶

Month three 7) Grenade system: explosion → cell edits + impulse; add timewarp VFX as an FBO post-pass (shockwave). ¹⁵

8) Hook time scaling to gameplay actions via the engine time spine (global slow-mo + local prototype). ³³

Month four 9) Sprite-sheet animation system with events; wire to audio. ⁵⁷

10) NPC v1: BT for decisions + A* pathfinding over a coarse occupancy grid derived from chunks. ⁴⁶

Month five 11) Save/load v1 for world chunks + entities, with a deterministic “round-trip no behavior change” test harness. ²⁴

12) AI debugging overlays (path, target, state) in-engine (optionally ImGui for tooling speed). ⁵⁸

Month six 13) Profiling pass: establish frame budgets; lock “no regression” thresholds in CI; add a performance capture mode. Epoch’s tooling docs already target scripting build/launch/capture/verify/close automation—extend that into CI where possible. ⁵⁹

14) Package a “vertical slice build” with one playable loop (spawn → dig → fight → timewarp grenade) and document it as the canonical 2D lane milestone.

Step-by-step task list for the two-year plan

Treat year two as “scale and harden,” not “reinvent.”

- **Quarter one-two:** deepen simulation (more materials/interactions), streaming, better collision extraction, robust save migrations. ⁴²
- **Quarter three:** NPC AI upgrades—incremental replanning (D* Lite) to handle destructible terrain, plus improved debugging. ⁶⁰
- **Quarter four:** performance scaling (GPU compute path if required), gameplay content expansion, distribution-quality packaging, and a public engine/game release cadence based on your existing versioning discipline. ⁶¹

Performance targets, profiling strategy, and desktop-first/mobile-optional guidance

Practical performance targets for your specific game

Because your game is simulation-heavy, your real target is not “max FPS,” but “stable frame time under load.” The archetype ECS literature explicitly emphasizes frame stability and cache efficiency as critical performance outcomes when scaling simulation workloads. ¹⁸

A reasonable desktop-first target for the six-month slice (aligned to your baseline hardware assumption) is:

- 60 FPS **with** pixel simulation, digging, AI, and post FX active in a moderate-sized play area (heuristic target).
- A “stress scene” that tests worst-case explosion + flowing materials without spiraling into >33ms frames.

Profiling stack that matches your engine's multi-backend reality

For CPU + GPU profiling in engine development, you want tools that work across “real backends” and integrate with your capture-driven discipline.

- **Tracy profiler** is explicitly built for real-time, nanosecond-resolution telemetry and supports profiling CPU + GPU across major graphics APIs including OpenGL/Vulkan/Direct3D/Metal, plus memory, locks, and context switches. ⁶²
- **RenderDoc** is widely used for single-frame GPU debugging across APIs including OpenGL/OpenGL ES/Vulkan, and is treated as a preferred call-analysis tool in industry guidance. ⁶³
- **Perfetto** is an open-source tracing suite built into Android (and useful on Linux) for correlating app traces with OS-level scheduling/memory/CPU data—ideal if you later pursue Android/mobile. ⁶⁴

Desktop-first, mobile-optional decision rule

Epoch already supports multiple backends and has a headless/noop concept. ⁵⁴
To stay sane as a solo dev:

- Ship the 2D lane on a single GPU backend first (OpenGL).
- Keep Vulkan as “build-only / isolated validation” until the core engine + game are stable, matching your own regression plan stance. ⁵²
- If mobile becomes required, evaluate whether OpenGL ES is acceptable or whether portability demands a stricter rendering abstraction (at that point, Vulkan/Metal become business decisions, not curiosity projects).

Methodology, QA automation, security, licensing, distribution, and cost/hardware requirements

Solo-friendly methodology that matches your existing workflow discipline

Your smoke/capture automation doc and commit pattern already resemble a high-functioning solo “agile variant”: small increments, versioned commits, visible proofs, regression discipline. ⁵¹

A solo-friendly cadence that works in practice:

- **Two-week iterations**, each ending with:
 - a playable build of the sandbox test project,
 - a capture set committed (or stored) for backend parity,
 - and a short “what changed, what broke, what’s next” note.
- **Monthly milestone**: one new “player-facing” capability (digging, grenade timewarp, NPC) plus one “engine hygiene” investment (save/load, profiler integration, CI capture run).

This matches your own docs’ emphasis on refreshing proofs every N versions and tying captures to source versions. ⁶⁵

QA and automated testing strategy for this engine

A rigorous solo QA strategy leans on determinism and automation rather than manual testing marathons:

- **Smoke scenarios per backend** (already documented) remain the backbone. ⁴³
- **Deterministic save/load tests** (Factorio-style) become your highest ROI automated test once chunks/simulation exist. ²⁴
- **Capture-based rendering regression tests** (already aligned with Epoch's capture flow) become your "visual unit tests." ⁵⁹

Security and supply chain practices appropriate for a solo engine

Even solo projects get hit by supply-chain and secret-leak issues. NIST's SSDF exists specifically because most SDLCs don't address security in detail and need practices integrated to reduce vulnerabilities and mitigate their impact. ⁶⁶

Practical, low-overhead actions:

- Treat third-party dependencies as liabilities: pin versions, record provenance, and avoid "mystery downloads." (General security best practice; SSDF supports adopting secure dev practices. ⁶⁷)
- If you use GitHub Actions, store secrets correctly and restrict permissions; GitHub documents that Actions secrets must be explicitly included and are encrypted client-side. ⁶⁸
- Consider signing release artifacts if/when you distribute binaries; Sigstore's Cosign workflow describes signing and verification with a transparency log. ⁶⁹

Licensing and distribution considerations for Epoch Engine as currently published

Your repository license is explicitly "LicenseRef-MIT-NoSell" and states that **commercial use is prohibited without prior written permission.** ⁷⁰

This has major implications:

- Under the Open Source Initiative ⁷¹ Open Source Definition, an open-source license must not discriminate against fields of endeavor, explicitly including business/commercial use. A "non-commercial only" restriction conflicts with that criterion. ⁷²
- SPDX license expressions explicitly allow "LicenseRef-*" identifiers as user-defined license references, which is consistent with your "LicenseRef-MIT-NoSell" labeling, but it also means downstream users must read your custom terms rather than relying on a standard OSI license label. ⁷³

If your long-term plan includes commercial distribution of games built on the engine, you may want a dual-licensing strategy (e.g., non-commercial for the public repo, commercial license available separately), but that is a business/legal decision—just don't call it "open source" if it prohibits commercial use, because OSI explicitly frames commercial use as guaranteed by the OSD. ⁷²

Hardware requirements and cost framing

Epoch's docs already define a baseline compatibility target ("6-core / GTX 1660 Ti-era desktops" and similar modern Linux laptops). ¹³

For the six-month lane, the most cost-effective hardware path is:

- One primary dev machine meeting that baseline,
- A second monitor (tooling productivity),
- Optional: a midrange Android device later if you pursue mobile profiling (to use Perfetto effectively).

⁷⁴

Because hardware pricing is volatile, treat dollar figures as "check current prices," but the capability requirements above are stable and aligned with your engine's stated support posture. ⁷⁵

Case studies and lessons that directly apply to your game + engine goals

- Noita ³⁰ provides a highly relevant case: marketed on pixel-level simulation, and its GDC talk description explicitly covers scaling falling-sand simulation to large continuous worlds and integrating destructible rigid-body physics—exactly the architectural problem you're tackling. ³¹
- Factorio ⁷⁶ is an exemplary case study for deterministic engineering discipline: its developers explain why deterministic save/load is required for multiplayer/replay and how they test for "no observable change" after reload. That mindset is directly applicable to your world simulation + save pipeline. ⁷⁷
- Long-running small-team projects like Dwarf Fortress ⁷⁸ demonstrate that continuous development over many years is viable when scope is managed and systems are evolved rather than rewritten—but it also underscores that "forever engine" thinking must be anchored by playable milestones, not architecture alone. ⁷⁹

¹ ² ¹² ¹³ ¹⁴ ¹⁷ ²¹ ⁵⁶ <https://github.com/Autodidac/EpochEngine>

<https://github.com/Autodidac/EpochEngine>

³ ⁵ ⁴⁷ ⁴⁹ ⁷⁵ https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/engine_analysis.md

https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/engine_analysis.md

⁴ ²⁰ ²⁸ <https://www.gdcvault.com/play/1025695/Exploring-the-Tech-and-Design/>

<https://www.gdcvault.com/play/1025695/Exploring-the-Tech-and-Design/>

⁶ ¹⁶ ⁵⁴ https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/aengineconfig_flags.md

https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/aengineconfig_flags.md

⁷ ⁴³ ⁴⁴ ⁵² ⁵³ ⁷¹ https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/renderer_regression_plan.md

https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/renderer_regression_plan.md

⁸ ¹⁰ ²³ ³³ ⁴⁰ ⁴¹ https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/runtime_operations.md

https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/runtime_operations.md

⁹ <https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/filelist.txt>

<https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/filelist.txt>

11 36 https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/ai_build_memory.md
https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/ai_build_memory.md

15 45 78 https://wikis.khronos.org/opengl/Framebuffer_Object
https://wikis.khronos.org/opengl/Framebuffer_Object

18 19 27 https://boyang.cs.uwm.edu/publication/sac2026_ECS.pdf
https://boyang.cs.uwm.edu/publication/sac2026_ECS.pdf

22 39 51 55 59 61 65 https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/smoke_capture_automation.md
https://github.com/Autodidac/EpochEngine/blob/main/Engine/docs/smoke_capture_automation.md

24 42 77 <https://www.factorio.com/blog/post/fff-270>
<https://www.factorio.com/blog/post/fff-270>

25 <https://miniaud.io/index.html>
<https://miniaud.io/index.html>

26 58 <https://github.com/ocornut/imgui>
<https://github.com/ocornut/imgui>

29 <https://www.khronos.org/opengl/wiki/Compute%20Shader>
<https://www.khronos.org/opengl/wiki/Compute%20Shader>

30 46 https://www.researchgate.net/publication/312869797_Behavior_Trees_for_Computer_Games
https://www.researchgate.net/publication/312869797_Behavior_Trees_for_Computer_Games

31 <https://store.steampowered.com/app/881100/Noita>
<https://store.steampowered.com/app/881100/Noita>

32 <https://www.youtube.com/watch?v=MxXATQCwRG0>
<https://www.youtube.com/watch?v=MxXATQCwRG0>

34 57 <https://github.com/Autodidac/EpochEngine/blob/main/Engine/modules/epochengine.ixx>
<https://github.com/Autodidac/EpochEngine/blob/main/Engine/modules/epochengine.ixx>

35 https://www.researchgate.net/publication/319463746_Behavior_Trees_in_Robotics_and_AI_An_Introduction
https://www.researchgate.net/publication/319463746_Behavior_Trees_in_Robotics_and_AI_An_Introduction

37 <https://gdcvault.com/play/1013282/Three-States-and-a-Plan>
<https://gdcvault.com/play/1013282/Three-States-and-a-Plan>

38 <https://cir.nii.ac.jp/crid/1363388844102264832?lang=en>
<https://cir.nii.ac.jp/crid/1363388844102264832?lang=en>

48 60 <https://www.ri.cmu.edu/publications/d-lite/>
<https://www.ri.cmu.edu/publications/d-lite/>

50 70 <https://github.com/Autodidac/EpochEngine/blob/main/LICENSE>
<https://github.com/Autodidac/EpochEngine/blob/main/LICENSE>

62 <https://github.com/wolfpld/tracy>
<https://github.com/wolfpld/tracy>

63 <https://developer.samsung.com/galaxy-gamedev/resources/tool-guides/renderdoc.html>
<https://developer.samsung.com/galaxy-gamedev/resources/tool-guides/renderdoc.html>

64 <https://perfetto.dev/docs/>
<https://perfetto.dev/docs/>

66 67 76 <https://csrc.nist.gov/pubs/sp/800/218/final>
<https://csrc.nist.gov/pubs/sp/800/218/final>

68 <https://docs.github.com/en/actions/concepts/security/about-secrets>
<https://docs.github.com/en/actions/concepts/security/about-secrets>

69 <https://docs.sigstore.dev/quickstart/quickstart-cosign/>
<https://docs.sigstore.dev/quickstart/quickstart-cosign/>

72 <https://opensource.org/definition/>
<https://opensource.org/definition/>

73 <https://spdx.github.io/spdx-spec/v3.0.1/annexes/spdx-license-expressions/>
<https://spdx.github.io/spdx-spec/v3.0.1/annexes/spdx-license-expressions/>

74 <https://developer.android.com/tools/perfetto>
<https://developer.android.com/tools/perfetto>

79 https://en.wikipedia.org/wiki/Dwarf_Fortress
https://en.wikipedia.org/wiki/Dwarf_Fortress